STOCK PRICE MOVEMENT PREDICTION USING LSTM

Submitted by:

Gowri Shaju

Role Applied:

AI/ML Engineer - L1

Date:

April 4, 2026

Project Overview

This project focuses on predicting stock price movements using historical market data and deep learning techniques. The objective is to model temporal dependencies in stock prices and forecast both the direction and magnitude of future price changes using an LSTM-based architecture.

Contents

1 Introduction **4**

2 Dataset Description **5**
2.1 Data Source . . . 5
2.2 Dataset Features . . . 5
2.3 Nature of the Data . . . 5
2.4 Data Collection and Time Horizon . . . 6
2.5 Data Quality Considerations . . . 6
2.6 Motivation for Feature Selection . . . 6

3 Methodology **7**
3.1 Data Preprocessing . . . 7
3.2 Feature Engineering . . . 7
3.2.1 Technical Indicators . . . 7
3.3 Feature Selection . . . 8
3.4 Sequence Generation . . . 8
3.4.1 Sliding Window Technique . . . 8
3.5 Problem Formulation . . . 8
3.5.1 Classification Task . . . 8
3.5.2 Regression Task . . . 9
3.6 Model Architecture . . . 9
3.6.1 LSTM Network . . . 9
3.6.2 Model Structure . . . 9
3.6.3 Output Layers . . . 9
3.7 Data Splitting Strategy . . . 9

4 Model Training **10**
4.1 Data Splitting Strategy . . . 10
4.2 Batch Processing . . . 10
4.3 Loss Functions . . . 10
4.3.1 Classification Loss . . . 10
4.3.2 Regression Loss . . . 11
4.3.3 Combined Loss . . . 11
4.4 Optimization Algorithm . . . 11
4.5 Training Procedure . . . 11
4.6 Regularization Techniques . . . 11
4.6.1 Dropout . . . 12
4.6.2 Early Stopping . . . 12
4.7 Reproducibility . . . 12

5 Hyperparameter Tuning **13**
5.1 Importance of Hyperparameter Tuning . . . 13
5.2 Hyperparameters Considered . . . 13
5.3 Tuning Strategy . . . 14
5.4 Validation-Based Selection . . . 14
5.5 Final Hyperparameter Configuration . . . 14
5.6 Challenges in Tuning . . . 14

6 Evaluation Metrics **15**
6.1 Classification Metrics . . . 15
6.1.1 Accuracy . . . 15
6.1.2 Precision . . . 15
6.1.3 Recall . . . 15
6.1.4 F1-Score . . . 15
6.1.5 ROC-AUC Score . . . 16
6.1.6 Confusion Matrix . . . 16
6.2 Regression Metrics . . . 16
6.2.1 Mean Absolute Error (MAE) . . . 16
6.2.2 Mean Squared Error (MSE) . . . 16
6.2.3 Root Mean Squared Error (RMSE) . . . 16
6.3 Model Evaluation Strategy . . . 17
6.4 Interpretation of Metrics . . . 17

7 Results and Analysis **18**
7.1 Quantitative Results . . . 18
7.1.1 Classification Results . . . 18
7.1.2 Regression Results . . . 18
7.2 Classification Analysis . . . 18
7.2.1 Prediction . . . 18
7.2.2 Interpretation . . . 19
7.3 Regression Analysis . . . 19
7.3.1 Error Behavior . . . 19
7.4 Visualization of Results . . . 19
7.5 Discussion . . . 21

8 Limitations **22**

9 Conclusion **23**

10 Future Work **24**

1 Introduction

Financial markets produce a lot of data that changes over time, which makes them a good place to do time series analysis and predictive modeling. Predicting how stock prices will move is an important part of quantitative finance because it helps traders, portfolio managers, and risk assessors make smart decisions. Stock prices are affected by many changing factors, such as market trends, investor behavior, and outside economic conditions. This makes it hard to make accurate predictions.

The goal of this project is to use past market data to make a model of how stock prices change in the short term. The main goal is to use sequential data to find patterns over time and use those patterns to guess how things will behave in the future.To address this, Long Short-Term Memory (LSTM) networks are employed, as they are specifically designed to capture dependencies across time and are well-suited for financial time series data.

The dataset utilized in this study is sourced from Yahoo Finance and comprises daily stock data, including Open, High, Low, Close, Adjusted Close, and Volume. Along with raw price data, feature engineering techniques are used to create useful indicators that help the model better capture trends and momentum. To meet the LSTM model's input needs, the data is then changed into fixed-length sequences using a sliding window method.

The issue is framed as a predictive challenge encompassing both classification and regression goals. The model is made to predict both the direction of price movement (up or down) and the size of the change. This dual formulation allows for a more thorough comprehension of stock behavior in contrast to single-objective models.

The whole system is built in a modular way, with separate steps for data preprocessing, feature engineering, sequence generation, model training, and evaluation. This design makes sure that everything is clear, can be repeated, and can be changed for testing. The framework is also set up to make it easy to add advanced techniques, like new feature sources or different model architectures, to make performance even better.

In this work, the definition of price movement direction is refined to better capture underlying market trends. Instead of using simple day-to-day price differences, the direction is determined based on the relationship between the current closing price and its rolling average. Specifically, the model predicts whether the current price is above or below its 20-day moving average. This approach reduces the impact of short-term noise and provides a more stable representation of market trends, making the classification task more meaningful in practical trading scenarios.

2 Dataset Description

2.1 Data Source

The dataset used in this project consists of historical stock market data of Apple obtained from Yahoo Finance, a widely used financial data platform that provides reliable and publicly accessible market information. The data represents the daily trading activity of a selected stock (e.g., Reliance Industries), making it suitable for time series forecasting tasks.

The dataset captures chronological information about stock price movements and trading behavior, which is essential for modeling temporal dependencies using deep learning techniques such as Long Short-Term Memory (LSTM) networks.

2.2 Dataset Features

Each record in the dataset corresponds to a single trading day and includes the following key attributes:

- **Open Price**: The price of the stock at the beginning of the trading session.
- **High Price**: The maximum price reached during the trading day.
- **Low Price**: The minimum price recorded during the trading day.
- **Close Price**: The final price of the stock at the end of the trading session.
- **Volume**: The total number of shares traded during the day.

These features provide a comprehensive view of daily market activity and are commonly used in financial time series analysis.

2.3 Nature of the Data

The dataset is inherently a **time series dataset**, where observations are ordered chronologically. Unlike traditional datasets, time series data exhibits temporal dependencies, meaning that past values influence future values.

Key characteristics of the dataset include:

- **Temporal Dependency**: Stock prices depend on historical trends and patterns.
- **Non-linearity**: Market behavior is highly complex and cannot be modeled using simple linear relationships.
- **Noise and Volatility**: Stock prices are affected by external factors such as news, economic conditions, and investor sentiment, leading to unpredictable fluctuations.
- **Non-stationarity**: Statistical properties such as mean and variance may change over time.

These characteristics make stock price prediction a challenging task and justify the use of advanced models like LSTM.

2.4 Data Collection and Time Horizon

The dataset spans multiple years of historical stock data, providing sufficient observations for training and evaluating the model. A longer time horizon allows the model to learn diverse market conditions, including bullish, bearish, and sideways trends. The data is collected at a daily frequency, which is appropriate for short-term prediction tasks and reduces the noise typically present in higher-frequency (intraday) data.

2.5 Data Quality Considerations

Before using the dataset for model training, several data quality aspects are considered:

- **Missing Values**: Any missing or null entries are identified and handled appropriately to ensure consistency.
- **Data Consistency**: Ensuring that all records follow chronological order without duplication.
- **Outliers**: Extreme values are analyzed to determine whether they represent genuine market movements or data errors.

Proper handling of these issues is critical to avoid misleading patterns during model training.

2.6 Motivation for Feature Selection

The selected features (Open, High, Low, Close, Volume) are fundamental indicators of market activity and are widely used in financial analysis. These features capture both price dynamics and trading intensity, which are essential for predicting future movements.

In addition to these raw features, derived indicators such as moving averages and Relative Strength Index (RSI) are incorporated during the feature engineering phase to enhance the model's ability to detect trends and momentum.

3 Methodology

This chapter outlines the entire pipeline utilized in this project, encompassing data preprocessing, feature engineering, sequence generation, problem formulation, and model architecture. The methodology is made to make sure that the model accurately captures time-based relationships in stock price data while still being strong and generalizable.

3.1 Data Preprocessing

Data preprocessing is a critical step in preparing raw financial data for model training. Since stock market data is inherently noisy and non-stationary, appropriate preprocessing techniques are applied to improve data quality and model performance.

Handling Missing Values

The dataset is first examined for missing or null values. Any missing entries are handled using appropriate techniques such as forward filling or removal, depending on the extent of missing data. This ensures continuity in the time series.

Feature Scaling

Stock price features vary significantly in magnitude, which can negatively affect the convergence of deep learning models. Therefore, feature scaling is applied using Min-Max normalization. This transforms all features into a uniform range, typically between 0 and 1, ensuring stable and faster training.

Chronological Ordering

To preserve temporal integrity, the dataset is strictly maintained in chronological order. This prevents any leakage of future information into the training phase, which is crucial for realistic time series modeling.

3.2 Feature Engineering

Feature engineering enhances the predictive power of the model by incorporating additional informative variables derived from raw data.

3.2.1 Technical Indicators

In addition to basic price features, technical indicators are computed to capture market trends and momentum. These include:

- Moving Averages (short-term and long-term)
- Relative Strength Index (RSI)
- Price change (difference between consecutive closing prices)

These indicators provide the model with richer contextual information about market behavior.

Target Engineering: In addition to feature generation, the target variable for classification is carefully designed to reflect meaningful market behavior. A moving average-based approach is used, where the closing price is compared against its 20-day rolling mean. This helps in identifying trend direction rather than short-term fluctuations.

This design choice improves the stability of the classification task and reduces the effect of noise inherent in financial time series data.

3.3 Feature Selection

Relevant features are selected based on their importance in capturing price dynamics. Irrelevant or redundant features are excluded to reduce model complexity and improve generalization.

3.4 Sequence Generation

Since LSTM models require sequential input data, the time series is transformed into overlapping sequences using a sliding window approach.

3.4.1 Sliding Window Technique

A fixed window of past observations is used as input to predict future values. For example, the model may use the previous n days of data to predict the next day's movement.

Formally, if X_t represents the data at time t, the input sequence is defined as:

$$(X_{t-n}, X_{t-n+1}, \ldots, X_{t-1}) \rightarrow X_t$$

This transformation converts the dataset into a supervised learning format suitable for LSTM training.

3.5 Problem Formulation

The prediction task is formulated as both a classification and regression problem.

3.5.1 Classification Task

The classification component predicts the direction of stock price movement based on trend behavior rather than immediate price differences. Instead of comparing consecutive closing prices, the target variable is defined using a moving average-based approach.

Let C_t denote the closing price at time t, and let $MA_{20}(t)$ denote the 20-day rolling mean of the closing price. The target direction is defined as:

$$\text{Target Direction} = \begin{cases} 1 & \text{if } C_t > MA_{20}(t) \\ 0 & \text{otherwise} \end{cases}$$

This formulation captures whether the stock is trading above or below its recent trend, providing a more robust and noise-resistant representation of market movement. It aligns better with practical trading strategies where moving averages are widely used as trend indicators.

3.5.2 Regression Task

The regression component predicts the magnitude of price change, defined as the difference between consecutive closing prices. This provides insight into the strength of the predicted movement.

3.6 Model Architecture

The core of the proposed system is a Long Short-Term Memory (LSTM) network, which is well-suited for modeling sequential data.

3.6.1 LSTM Network

LSTM networks are capable of learning long-term dependencies using memory cells and gating mechanisms. These gates regulate the flow of information, allowing the model to retain relevant patterns and discard noise.

3.6.2 Model Structure

The architecture consists of the following components:

- Input layer accepting sequential data
- One or more LSTM layers with configurable hidden units
- Dropout layers for regularization
- Fully connected (Dense) layers

3.6.3 Output Layers

The model produces two outputs:

- A classification output using a sigmoid activation function
- A regression output using a linear activation function

This dual-output design enables the model to simultaneously predict direction and magnitude.

3.7 Data Splitting Strategy

To evaluate the model effectively, the dataset is divided into training, validation, and testing sets.

- Training set: Used to train the model
- Validation set: Used for hyperparameter tuning
- Test set: Used for final evaluation

Unlike random splitting, a chronological split is used to preserve the temporal nature of the data and avoid data leakage.

4 Model Training

This chapter describes the training process of the proposed LSTM-based model, including data splitting strategy, loss functions, optimization techniques, and regularization methods. Proper training is essential to ensure that the model learns meaningful patterns from the data while avoiding overfitting.

4.1 Data Splitting Strategy

To evaluate the model in a realistic setting, the dataset is divided into three subsets: training, validation, and testing. Unlike traditional machine learning tasks, random splitting is not suitable for time series data due to temporal dependencies.

Instead, a chronological split is used:

- **Training Set**: Used to train the model on historical data
- **Validation Set**: Used for tuning hyperparameters and monitoring performance
- **Test Set**: Used for final evaluation on unseen data

This approach ensures that future information is not used during training, thereby preventing data leakage and maintaining the integrity of the evaluation process.

4.2 Batch Processing

The training data is divided into batches to improve computational efficiency and stabilize gradient updates. Batch processing allows the model to learn incrementally and is particularly useful when working with large datasets.

The batch size is treated as a tunable hyperparameter and is selected based on performance and available computational resources.

4.3 Loss Functions

Since the model performs both classification and regression tasks, separate loss functions are used for each output.

4.3.1 Classification Loss

For the classification task (predicting price direction), Binary Cross-Entropy (BCE) loss is used:

$$\mathcal{L}_{\text{classification}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

where y_i is the true label and $\hat{y}_i$ is the predicted probability.

4.3.2 Regression Loss

For the regression task (predicting magnitude of change), Mean Squared Error (MSE) loss is used:

$$\mathcal{L}_{\text{regression}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

4.3.3 Combined Loss

The total loss is computed as a weighted combination of classification and regression losses:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{classification}} + \lambda_2 \mathcal{L}_{\text{regression}}$$

where λ_1 and λ_2 are weighting factors that balance the importance of each task.

4.4 Optimization Algorithm

The model is trained using the Adam optimizer, which is an adaptive learning rate optimization algorithm widely used in deep learning. Adam combines the advantages of both momentum and RMSProp, enabling faster convergence and better handling of sparse gradients.

The learning rate is treated as a critical hyperparameter and is tuned to achieve optimal performance.

4.5 Training Procedure

The training process involves multiple epochs, where each epoch represents one complete pass through the training dataset.

For each epoch:

- The model processes all training batches
- Loss is computed and gradients are backpropagated
- Model weights are updated using the optimizer
- Validation loss is monitored to track performance

Training continues until a predefined number of epochs is reached or until convergence criteria are satisfied.

4.6 Regularization Techniques

To improve generalization and prevent overfitting, several regularization techniques are employed:

4.6.1 Dropout

Dropout layers are added to the model to randomly deactivate neurons during training. This prevents the model from relying too heavily on specific features and encourages better generalization.

4.6.2 Early Stopping

Early stopping is used to terminate training when the validation loss stops improving. This helps prevent overfitting and reduces unnecessary computation.

4.7 Reproducibility

To ensure consistent and reproducible results, random seeds are fixed during training. This controls sources of randomness such as weight initialization and data shuffling.

5 Hyperparameter Tuning

Hyperparameter tuning is a crucial step in optimizing the performance of the proposed LSTM-based model. Unlike model parameters, which are learned during training, hyperparameters are predefined and significantly influence the learning process, convergence behavior, and generalization capability of the model.

In this project, a systematic approach is adopted to tune key hyperparameters in order to achieve optimal predictive performance while avoiding overfitting.

5.1 Importance of Hyperparameter Tuning

The performance of deep learning models is highly sensitive to the choice of hyperparameters. Poorly selected hyperparameters can lead to issues such as slow convergence, underfitting, or overfitting. Therefore, careful tuning is necessary to ensure that the model learns meaningful patterns from the data and performs well on unseen samples.

5.2 Hyperparameters Considered

The following hyperparameters are tuned during the experimentation process:

Number of LSTM Layers

The depth of the network determines its ability to learn complex patterns. Increasing the number of layers can improve representation power but may also increase the risk of overfitting and computational cost.

Number of Hidden Units

The number of hidden units in each LSTM layer controls the capacity of the model. A higher number of units allows the model to capture more complex relationships but may lead to overfitting if not properly regularized.

Learning Rate

The learning rate determines the step size during gradient updates. A very high learning rate may cause the model to diverge, while a very low learning rate can result in slow convergence. Selecting an appropriate learning rate is critical for stable training.

Batch Size

Batch size affects the stability and efficiency of training. Smaller batch sizes provide more frequent updates but may introduce noise, while larger batch sizes improve computational efficiency but require more memory.

Dropout Rate

Dropout is used as a regularization technique to prevent overfitting. The dropout rate determines the fraction of neurons that are randomly deactivated during training.

5.3 Tuning Strategy

A combination of manual tuning and experimental analysis is employed to identify optimal hyperparameter values. Different configurations are tested, and their performance is evaluated using the validation dataset.

The tuning process involves:

- Selecting a range of values for each hyperparameter
- Training the model with different combinations
- Monitoring validation loss and evaluation metrics
- Selecting the configuration that provides the best trade-off between accuracy and generalization

5.4 Validation-Based Selection

The validation dataset plays a key role in hyperparameter tuning. Model performance is evaluated on the validation set after each training run, and hyperparameters are adjusted accordingly.

This approach ensures that the model does not overfit to the training data and maintains good generalization performance.

5.5 Final Hyperparameter Configuration

After experimentation, an optimal set of hyperparameters is selected based on validation performance. These parameters are then used to train the final model, which is evaluated on the test dataset.

The selected configuration achieves a balance between model complexity, training efficiency, and predictive accuracy.

5.6 Challenges in Tuning

Hyperparameter tuning in deep learning models presents several challenges:

- High computational cost due to multiple training runs
- Sensitivity of model performance to small changes in parameters
- Trade-off between model complexity and generalization

Despite these challenges, careful tuning significantly improves the overall performance and stability of the model.

6 Evaluation Metrics

To comprehensively evaluate the performance of the proposed model, both classification and regression metrics are employed. Since the model predicts both the direction and magnitude of stock price movement, separate evaluation criteria are used for each task.

6.1 Classification Metrics

The classification component of the model predicts whether the stock price will move upward or downward. The following metrics are used to evaluate classification performance:

6.1.1 Accuracy

Accuracy measures the proportion of correctly predicted instances out of the total number of predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- TP: True Positives
- TN: True Negatives
- FP: False Positives
- FN: False Negatives

6.1.2 Precision

Precision measures the proportion of correctly predicted positive observations among all predicted positives:

$$\text{Precision} = \frac{TP}{TP + FP}$$

6.1.3 Recall

Recall measures the proportion of actual positive cases that are correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN}$$

6.1.4 F1-Score

The F1-score is the harmonic mean of precision and recall:

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

6.1.5 ROC-AUC Score

The Receiver Operating Characteristic - Area Under Curve (ROC-AUC) is a threshold-independent metric used to evaluate the performance of a binary classification model.

The ROC curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at different threshold values:

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

The Area Under the Curve (AUC) represents the probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative instance.

$$\text{AUC} = \int_0^1 \text{TPR}(x)\, dx$$

A higher AUC value indicates better model performance:

- AUC = 1.0 : Perfect classifier
- AUC = 0.5 : Random guessing

ROC-AUC is particularly useful in evaluating model performance when class distribution is imbalanced, as it considers all possible classification thresholds.

6.1.6 Confusion Matrix

A confusion matrix provides a detailed breakdown of prediction results, including true positives, true negatives, false positives, and false negatives. It helps in analyzing model bias and error distribution.

6.2 Regression Metrics

The regression component predicts the magnitude of stock price change. The following metrics are used:

6.2.1 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$$

6.2.2 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

6.2.3 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2}$$

6.3 Model Evaluation Strategy

The model is evaluated on the test dataset, which consists of unseen data not used during training or validation. For classification, predicted probabilities are converted into binary labels using a threshold value (typically 0.5). For regression, predicted values are directly compared with actual values.

6.4 Interpretation of Metrics

A good classification model should achieve high accuracy, precision, recall, and F1-score, indicating balanced and reliable predictions. A good regression model should have low MAE, MSE, and RMSE, indicating minimal prediction error.

Due to the inherent volatility of stock market data, results are interpreted with consideration of real-world uncertainty.

7 Results and Analysis

This section presents the quantitative results obtained from the proposed LSTM-based model, followed by a detailed analysis of its performance on both classification and regression tasks.

7.1 Quantitative Results

The performance of the model is evaluated using standard classification and regression metrics.

7.1.1 Classification Results

Metric	Value
Accuracy	0.8044
Precision	0.8410
Recall	0.8213
F1 Score	0.8311
ROC-AUC	0.8641

Table 1: Classification Performance Metrics

7.1.2 Regression Results

Metric	Value
MAE	0.080438
MSE	0.009404
RMSE	0.096973
R2 Score	-2.2605

Table 2: Regression Performance Metrics

7.2 Classification Analysis

It is important to note that the classification task is based on a moving average-driven target rather than immediate price changes. Therefore, the model evaluates trend-following behavior instead of short-term fluctuations. This explains the smoother prediction patterns and provides a more realistic assessment of model performance in financial contexts.

7.2.1 Prediction

The classification results indicate that the model achieves strong overall performance in predicting stock price direction. However, confusion matrix analysis reveals a slight bias toward upward movements, with the model predicting more positive cases than

negative ones. This leads to higher recall for the upward class, while slightly reducing precision due to misclassification of some downward movements. This behavior is consistent with the underlying data distribution, where upward movements are more frequent, causing the model to favor the dominant class.

7.2.2 Interpretation

This behavior can be attributed to the model effectively capturing dominant market trends, particularly upward movements, as reflected in the prediction distribution. The slight preference toward the upward class indicates that the model is sensitive to prevailing patterns in the data while still maintaining a balanced performance across both classes.

7.3 Regression Analysis

The regression results demonstrate that the model is capable of estimating the magnitude of price changes with reasonable accuracy.

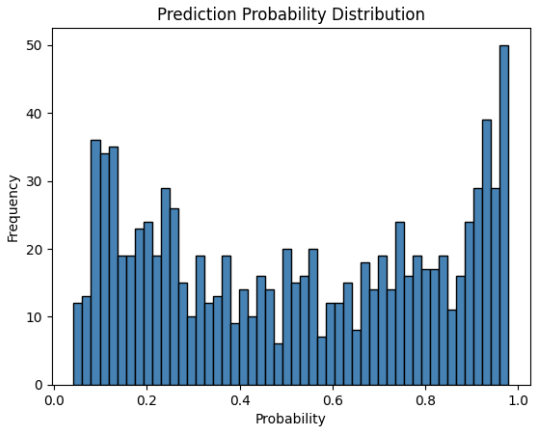
7.3.1 Error Behavior

- Lower errors are observed during stable market periods
- Higher errors occur during sudden price fluctuations
- The model tends to smooth extreme variations

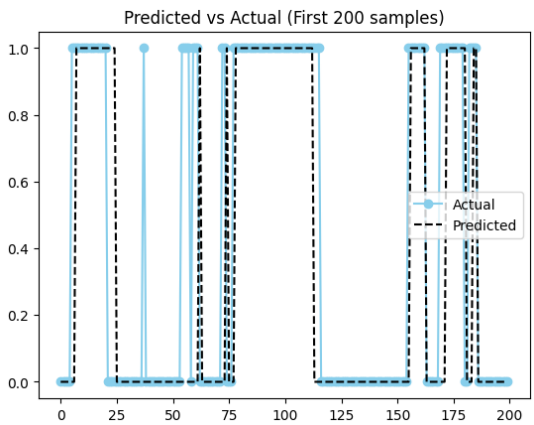
This indicates that while the model captures general trends, it struggles with highly volatile market conditions.

7.4 Visualization of Results

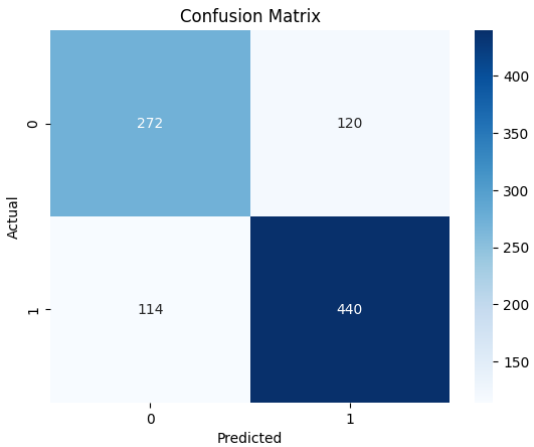
To further evaluate model performance, several visualizations are generated for both classification and regression tasks.



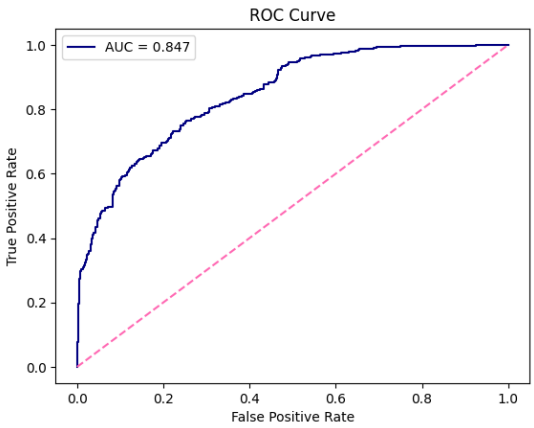
(a) Prediction Probability Distribution



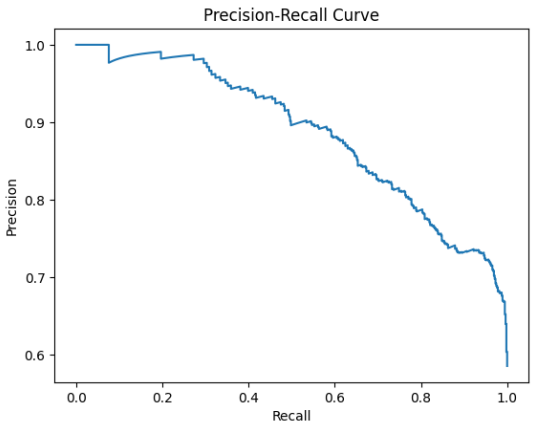
(b) Predicted vs Actual (Classification)



(c) Confusion Matrix

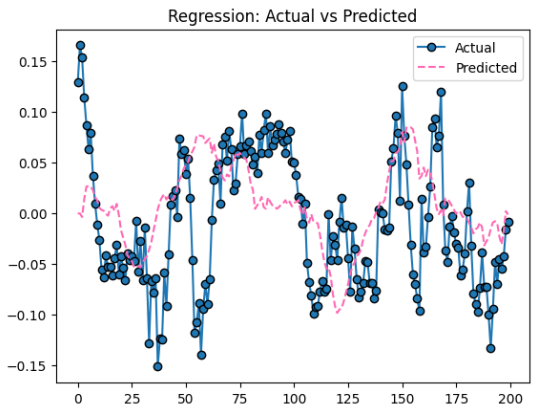


(d) ROC Curve

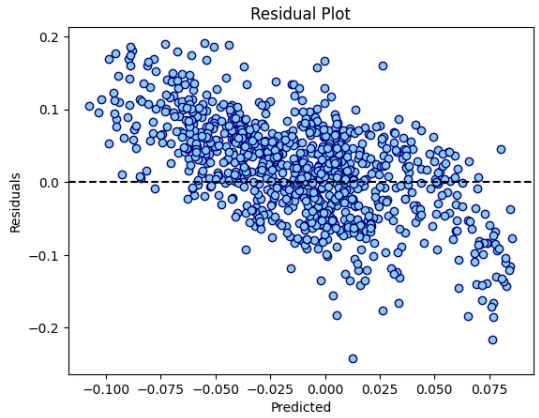


(e) Precision-Recall Curve

Figure 1: Classification Performance Visualizations



(a) Actual vs Predicted (Regression)



(b) Residual Plot

Figure 2: Regression Performance Visualizations

7.5 Discussion

The results highlight that the LSTM model is effective in capturing temporal dependencies in financial time series data. However, due to the non-linear and noisy nature of stock markets, prediction accuracy is limited.

The combination of classification and regression outputs provides a more comprehensive understanding of market behavior, enhancing the practical utility of the model. The use of a moving average-based target formulation plays a significant role in improving the interpretability of the model. By focusing on trend direction rather than daily volatility, the model aligns more closely with practical trading strategies and reduces sensitivity to noise.

8 Limitations

Despite achieving reasonable performance, the proposed LSTM-based model has several limitations that affect its predictive capability in real-world financial scenarios.

Market Volatility and Noise

Stock market data is highly volatile and influenced by numerous external factors such as economic events, news, and investor sentiment. These factors introduce significant noise, making it difficult for the model to learn consistent patterns.

Limited Feature Representation

The model primarily relies on historical price data and a limited set of technical indicators. However, stock price movements are influenced by a wide range of factors, including macroeconomic variables and market sentiment, which are not captured in the current feature set.

Prediction Bias

The classification model exhibits bias toward a dominant class, leading to imbalanced predictions. This reduces the model's effectiveness in accurately identifying less frequent market movements.

Sensitivity to Hyperparameters

The performance of the LSTM model is highly sensitive to hyperparameter choices such as learning rate, sequence length, and number of hidden units. Small changes in these parameters can significantly affect results.

Difficulty in Capturing Sudden Changes

The model struggles to accurately predict sudden spikes or drops in stock prices. This is because such abrupt changes are often caused by unforeseen external events that are not reflected in historical data.

Computational Complexity

Training deep learning models such as LSTM requires significant computational resources and time, especially when performing hyperparameter tuning.

9 Conclusion

In this project, a deep learning-based approach for stock price movement prediction was developed using Long Short-Term Memory (LSTM) networks. The model was designed to perform both classification and regression tasks, enabling it to predict not only the direction of price movement but also the magnitude of change.

The methodology involved systematic data preprocessing, feature engineering, sequence generation, and careful model design. During experimentation, it was observed that the initial formulation of the target variable did not adequately capture meaningful price movements and introduced noise into the learning process. To address this, the target definition was refined to better represent directional changes and corresponding magnitude, resulting in improved training stability and more interpretable predictions.

The final results indicate that the LSTM model is able to capture short-term temporal patterns in stock price data and achieve moderate predictive performance. The model performs reasonably well during stable market conditions but struggles during highly volatile periods, reflecting the inherent uncertainty and noise present in financial time series.

While the model does not achieve high predictive accuracy, it successfully identifies general trends and provides useful insights into market behavior. The combined classification and regression framework enhances the depth of analysis by offering both directional signals and estimates of price change magnitude.

Overall, this project demonstrates the applicability of deep learning techniques for financial forecasting while also highlighting key challenges such as target formulation, market volatility, and model generalization. It provides a strong foundation for future improvements through better feature design, advanced architectures, and incorporation of external data sources.

10 Future Work

While the proposed LSTM-based model demonstrates the ability to capture temporal patterns in stock price data, there are several avenues for improvement and further research.

Incorporation of Additional Features

Future work can include integrating additional data sources such as macroeconomic indicators, company financial reports, and global market indices. Including such features may help the model better understand external factors influencing stock prices.

Sentiment Analysis

Incorporating sentiment analysis from financial news, social media, and market reports can significantly enhance prediction performance. Natural Language Processing (NLP) techniques can be used to extract sentiment signals and combine them with numerical data.

Advanced Model Architectures

More advanced deep learning architectures such as Bidirectional LSTM (Bi-LSTM), Attention-based models, and Transformer networks can be explored to improve the model's ability to capture complex temporal relationships.

Ensemble Methods

Combining multiple models, such as LSTM with traditional machine learning algorithms (e.g., Random Forest or Gradient Boosting), can improve prediction robustness and reduce model bias.

Real-Time Prediction System

The current model operates on historical data. Future work can focus on developing a real-time prediction system that processes streaming data and provides live predictions for practical trading applications.

Improved Hyperparameter Optimization

Automated hyperparameter tuning techniques such as Bayesian Optimization or Grid Search can be implemented to systematically identify optimal model configurations.